U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, Thomas Brox (2015)

Reviewed by BeomSoo Son

Abstract

Problem

Deep networks were widely believed to require many thousands of annotated training samples.

Biomedical constraint

Segmentation labels usually require expert pixel-level annotation, so large labeled datasets are hard to obtain.

U-Net proposal

A network architecture and training strategy that learns end-to-end from very few annotated images.

Key mechanism

Contracting path captures context + Expansive path enables precise localization + Strong data augmentation

From Classification to Segmentation

Typical CNN use

Given an image, output one class label for the whole image.

Biomedical image use

One image often contains many cells or tissue structures, so a single label is not enough.

Required output

Assign a class label to every pixel: foreground, membrane, background, or task-specific classes.

Segmentation = Classification + Localization

Sliding Window

Earlier approach

Predict each pixel by feeding a local patch around that pixel into a CNN.

Advantage

The training set becomes much larger because patches, not whole images, become training examples.

Drawback 1 — Redundant computation

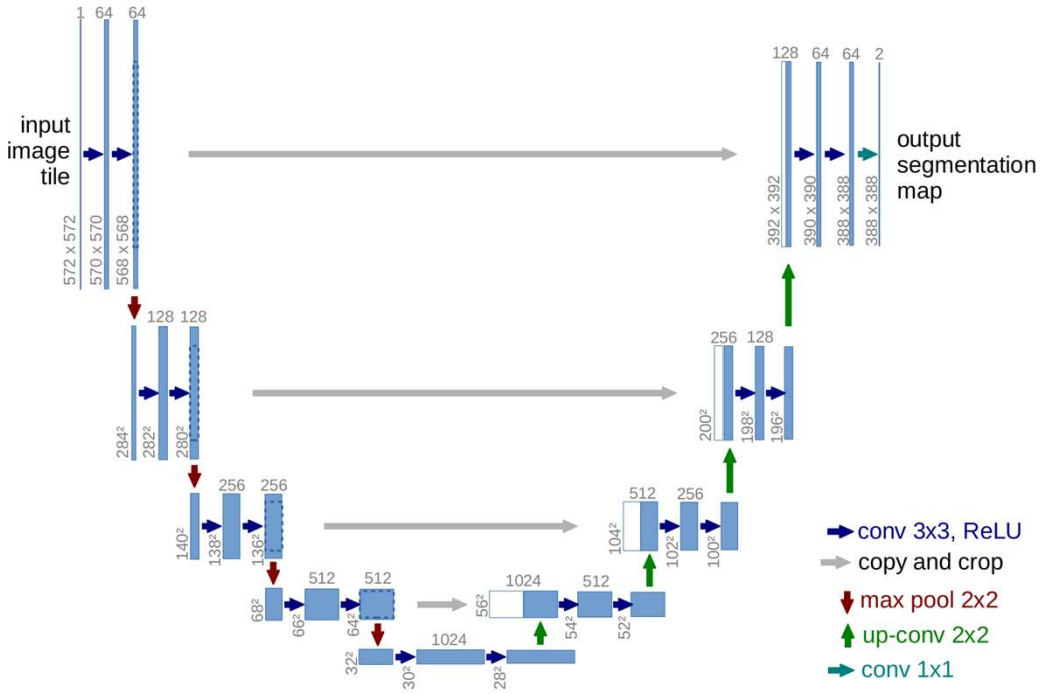
Overlapping patches are processed again and again, making inference slow.

Drawback 2 — Context/localization trade-off

Large patches see more context but lose localization after pooling; small patches localize but lack context.

But, U-Net avoids running the network separately for every patch by using a fully convolutional, tile-based strategy.

Overall Architecture



Contracting path

Repeated convolutions and max-pooling extract increasingly abstract context.

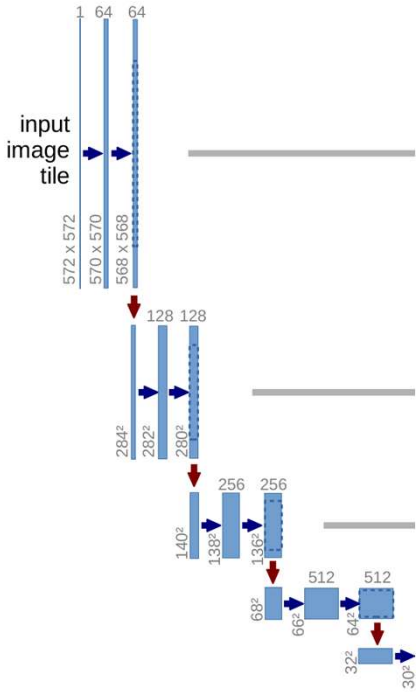
Expansive path

Upsampling restores spatial resolution for pixel-level localization.

Skip connections

High-resolution features are copied and cropped, then concatenated with upsampled features.

Contracting Path



Feature extraction

Two 3×3 valid convolutions, each followed by ReLU.

Downsampling

A 2×2 max-pooling operation with stride 2 halves width and height.

Channel expansion

At each downsampling step, the number of feature channels doubles.

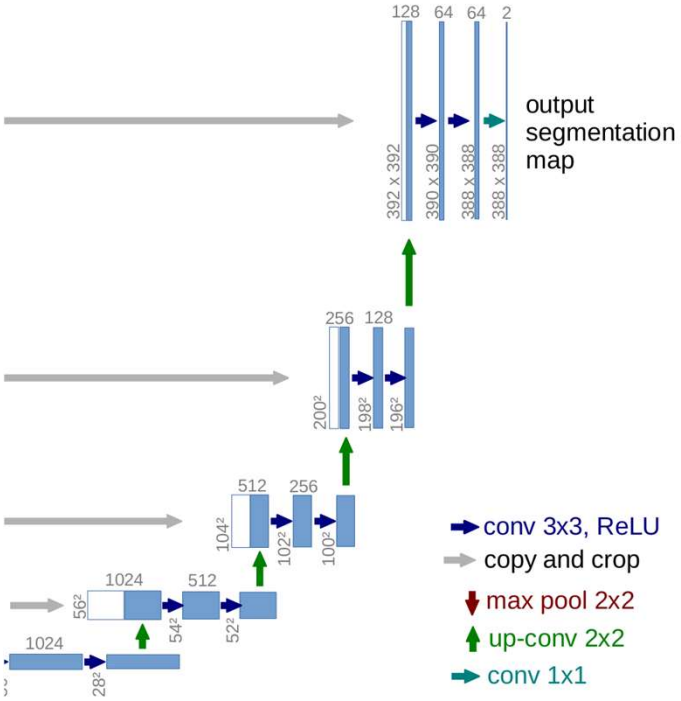
Why this matter?

The receptive field grows, allowing the network to understand larger context around each pixel.

[1, 2, 3, 4, 5]

[1, 2, 3] [2, 3, 4], [3, 4, 5]

Expansive Path



Upsampling

Each step upsamples the feature map, then uses a 2×2 up-convolution that halves the channel count.

Copy and crop

The corresponding feature map from the contracting path is cropped and concatenated.

Refinement

Two 3×3 valid convolutions with ReLU combine localization details with semantic context.

Final prediction

A 1×1 convolution maps each 64-component feature vector to the desired number of classes.

Valid Convolution & Border Issue

No padding

U-Net uses only the valid part of every convolution, so feature maps shrink after each convolution.

Input vs. output size

In the paper’s example, a 572×572 input tile produces a 388×388 output segmentation map.

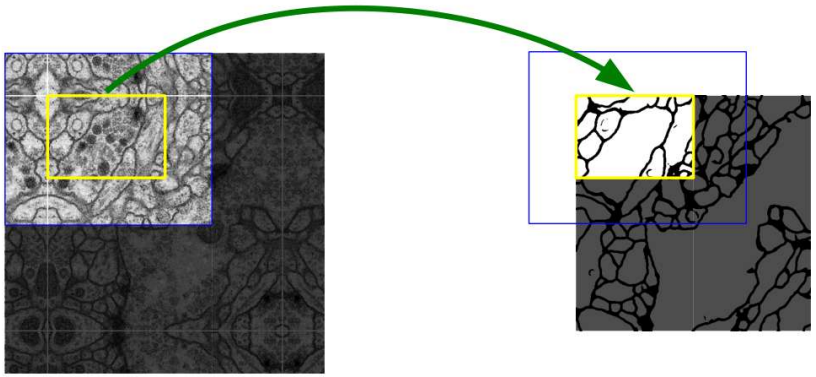
Reason

The output contains only pixels for which the full input context is available.

Consequence

Border regions need additional context; otherwise the network would miss information near image edges.

Overlap-Tile Strategy & Mirroring



Overlap-tile

Large images are split into tiles, but neighboring tiles overlap so every output pixel has enough context.

Mirroring

When context is missing at the image border, the input image is extended by reflecting it.

Effect

The model can segment arbitrarily large images while staying within GPU memory limits.

Training Setup

Optimizer

Stochastic gradient descent (SGD) implemented in Caffe.

Mini-batch strategy

Because valid convolutions shrink the output, U-Net favors large input tiles over large batch size; batch size is reduced to one image.

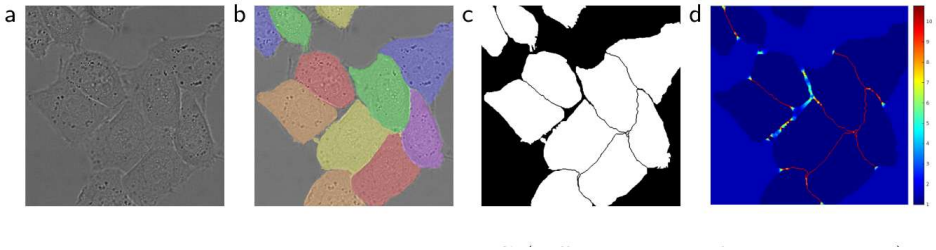
Momentum

Momentum is set to 0.99 so many previously seen samples influence the current update.

Initialization

Weights are drawn with standard deviation $\sqrt{(2/N)}$, helping feature maps keep approximately unit variance.

Weighted Loss for Cell Borders



$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

Problem

Touching cells can be merged if the model does not learn the thin separating borders.

Weighted cross-entropy

Each pixel receives a weight, so mistakes on important pixels contribute more to the loss.

Weight map

wc balances class frequency; d1 and d2 measure distances to the nearest and second-nearest cell borders.

Data Augmentation

Geometric invariance

Shift and rotation augmentation teach the model not to depend on exact cell position or orientation.

Gray-value robustness

Microscopy images have limited color variation, but intensity changes still matter.

Elastic deformation

The key idea: simulate realistic tissue deformation using random displacement vectors on a coarse 3×3 grid.

Implicit augmentation

Dropout layers at the end of the contracting path add further robustness.

Experiments: EM Segmentation Challenge

Task

Segment neuronal structures from electron microscopy stacks.

Training data

Only 30 fully annotated 512×512 images were used.

Evaluation

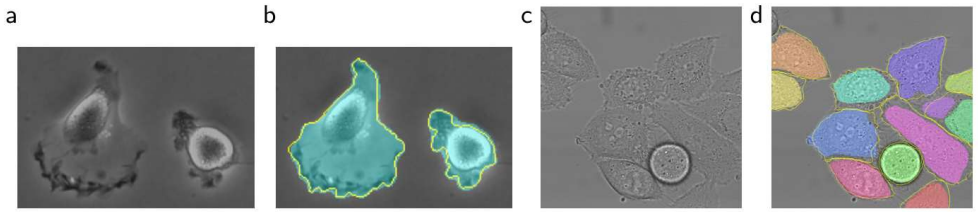
Predicted membrane probabilities are thresholded and judged by warping error, Rand error, and pixel error.

Rank 1
by warping error

0.000353
warping error

0.0382
Rand error

Experiments: ISBI Cell Tracking Challenge



Task

Segment cells in phase contrast and DIC light microscopy images.

Metric

IOU: how much the predicted cell area overlaps with the ground truth.

92.03%

PhC-U373 IOU
second-best: 83%

77.56%

DIC-HeLa IOU
second-best: 46%

Conclusion

Main achievement

U-Net delivers strong biomedical segmentation performance across different microscopy tasks.

Why it works with little data

Strong data augmentation, especially elastic deformation, makes limited annotations much more useful.

Why the architecture matters

The contracting path captures context while the expansive path and skip connections recover precise localization.

Practicality

The paper reports training in about 10 hours on an NVIDIA Titan GPU with 6 GB memory.

Thank You